

STUDENT PROJECT BOOK

C++ Foundations Project Book

Extended Activities & Capstone Challenges — Part 1 Companion

Introduction to C++ Programming — Part 1: Foundations

How to Use This Book

This book is a companion to Part 1 of the Introduction to C++ Programming course (Modules 0-8). It does not re-teach the lessons — read each module's README, work through its examples, and complete its own exercise and project first. What follows here are extra, independent activities for each topic: more practice, applied to new situations, with no starter code and no solutions provided.

If you are new to programming, that last part may feel uncomfortable — that is normal, and it is exactly how programming is learned. Every activity in this book uses **ONLY** what you have learned up to that chapter. Each chapter opens with a 'Use only' line that tells you exactly which tools are allowed. If you find yourself reaching for something the course hasn't taught yet, stop: there is a simpler way using what you already know, and finding it is the real exercise.

Each chapter offers three activities of increasing difficulty — **FOUNDATIONAL**, **CORE** and **CHALLENGE**. **FOUNDATIONAL** checks that you understood the lesson, **CORE** makes you apply it to a new problem, and **CHALLENGE** stretches you. You do not have to finish the **CHALLENGE** before moving on to the next module. Each **CHALLENGE** includes one hint: read it only after a genuine attempt. Every chapter ends with an optional domain twist that frames the same idea in an Electrical/Electronic Engineering or Biomedical Engineering setting — a preview of the engineering tracks you will meet in Part 2. The final chapter is an Extended Capstone — a bigger, optional challenge that goes beyond Module 9's own capstone.

When you get stuck: read the **FIRST** compiler error and look at the line it names (and the line just above it); compile after every few lines rather than at the end; print your variables with `std::cout` to see what the program is really doing; and explain your code out loud, line by line, to a classmate or even to an empty chair. Try for at least 15 minutes before asking for help — and when you do ask, show the exact error message and what you have already tried.

Commit every activity to your course GitHub repository, in a folder such as `project-book/module-03/`, exactly as Module 0 taught you: `git status`, `git add`, `git commit` with a clear message, `git push`. By the end of Part 1 your GitHub profile should show not only the course projects, but a whole book's worth of your own extra work.

Chapter 0 — Tools: VS Code, a C++ Compiler, Git & GitHub

Recap: The compiler (g++) turns your .cpp source file into a program you can run. VS Code is where you write and organise your code. Git records save points (commits) of your work on your own computer, and GitHub keeps a copy online. The daily workflow is status → add → commit → push, with a short, clear commit message that says what changed.

Use only: the terminal, g++, VS Code, and the Git commands from Module 0 (git status, git add, git commit, git push). No new C++ is needed — whenever an activity needs a program, copy the hello.cpp program from the lesson.

Activity 1 — Your Project-Book Home

DIFFICULTY: FOUNDATIONAL

Scenario: *Every activity in this book will be saved in one place. Set that place up properly now, once, and every later chapter becomes easier.*

Requirements:

- Inside your course folder, create a folder named project-book, and inside it a folder named module-00.
- Copy hello.cpp from the lesson into project-book/module-00/, then compile it with g++ from the terminal and run it.
- Create a file project-book/README.md in VS Code and write two or three sentences of your own in it: who you are, and what this folder will hold. (A .md file is just plain text.)
- Commit and push both files, then open your repository on GitHub.com and confirm that both files appear.

Activity 2 — Commits That Tell a Story

DIFFICULTY: CORE

Scenario: *A good commit history reads like a diary of how a project grew. A bad one ('update', 'stuff', 'fix') tells nobody anything — including you, a month from now.*

Requirements:

- Make three separate, small changes to your copy of hello.cpp — for example: change the message; add a second line of output; add a comment at the top with your name and the date. Compile and run after each change.
- Commit after EACH change — three commits in total — each with a message that says what changed, e.g. "Add second greeting line".
- Use git status before every git add, and write down in project-book/module-00/NOTES.md what git status showed you each time (which files were new or modified).
- Push, then find your three commits on GitHub.com by clicking the commits link on your repository page. Check that they appear newest first, with your messages.

Activity 3 — The VS Code Way

DIFFICULTY: CHALLENGE

Scenario: Module 0 showed two ways to save your work to GitHub: typing Git commands in the terminal, and clicking buttons in VS Code's Source Control panel. A professional is comfortable with both — and knows they do exactly the same thing.

Requirements:

- Make one more small change to hello.cpp, then commit and push it using ONLY the Source Control panel in VS Code — no terminal commands.
- Make another change and commit and push it using ONLY the terminal.
- In NOTES.md, write a small two-column table matching each terminal command (git status, git add, git commit, git push) to the button or view in VS Code that does the same job.
- Confirm on GitHub.com that both commits arrived, and add one sentence to NOTES.md saying which way you prefer and why.

Hint (read after a real attempt): The Source Control panel is the icon on VS Code's left-hand bar that looks like a branching line. Files listed under 'Changes' are what git status would show; the + next to a file is git add.

Domain Twist (optional)

EE twist: in your project-book README.md, add a short section listing the parts of a simple torch circuit (battery, switch, bulb, wires), and commit it as its own commit with a clear message. Biomedical twist: add a short section listing the steps a nurse follows when taking a patient's temperature, and commit it the same way — practising clear, specific commit messages on non-code changes too.

Chapter 1 — Basics of Programming

Recap: A program is a list of instructions the computer follows from top to bottom. The compiler checks your C++ source and translates it into a program you can run. Every C++ program starts in `int main();` `std::cout <<` prints to the screen; every statement ends with a semicolon; and comments (`//` and `/* */`) are notes for humans that the compiler ignores. Error messages are normal: read the first one, go to the line it names, and fix one thing at a time.

Use only: `#include <iostream>`, `int main()`, `std::cout` with `<<` and `std::endl`, text inside double quotes, comments, and `return 0`. Not yet: variables (Module 2) or reading input (Module 3) — every program prints fixed text.

Activity 1 — My Weekly Timetable

DIFFICULTY: FOUNDATIONAL

Scenario: *You want a quick reminder of your school week every time you open the terminal.*

Requirements:

- Print a timetable for Monday to Friday, one line per day, showing at least one lesson or activity each day (e.g. Monday | Maths, Coding Club).
- Print a title line above the timetable, underlined with a line of = or - characters.
- Use spaces so that the | characters line up in a straight column. Count carefully.
- Add a comment above each part of your output explaining what it prints.

Activity 2 — Shop Receipt

DIFFICULTY: CORE

Scenario: *A small provisions shop wants to see what a printed receipt could look like before it buys a till.*

Requirements:

- Print a receipt for at least four items, each on its own line, with the item name on the left and the price on the right (e.g. Bread GHS 15.00). The prices are just text inside quotes for now — you will calculate with real numbers in Module 2.
- Include the shop's name, a date, a line for the total, and a thank-you message at the bottom.
- Use at least one `std::cout` statement that chains several pieces of text together with `<<`.
- Put a multi-line `/* */` comment at the top of the file describing what the program does and who wrote it.

Activity 3 — Error Detective

DIFFICULTY: CHALLENGE

Scenario: Professional programmers see error messages every day. The skill is not avoiding them — it is reading them calmly. Practise that now, while your programs are still small.

Requirements:

- Copy your Activity 1 program into a new file, `detective.cpp`. Introduce ONE mistake at a time, compile, and write down the first line of the message the compiler gives. Try at least these five: delete a semicolon; misspell `cout` as `cot`; delete the closing `}`; remove the `#include <iostream>` line; and put single quotes instead of double quotes around a word.
- Fix each mistake and recompile before you introduce the next one.
- Record your findings in `project-book/module-01/ERRORS.md`: the mistake, the first line of the message, and the line number the compiler reported. Mark every case where the reported line was NOT the line with the mistake.
- The single-quote mistake is special: does the compiler give an error, or only a warning? What does the program print? Explain what you saw in `ERRORS.md`.

Hint (read after a real attempt): When a semicolon is missing, the compiler usually reports the problem on the NEXT line — it only notices something is wrong when the next statement begins. So always check the line named in the message AND the line just above it.

Domain Twist (optional)

*EE twist: print the label for an electronics kit — a parts list of resistors, LEDs, a battery holder and jumper wires, with quantities, laid out in neat columns. Biomedical twist: print a clinic's daily notice board — opening hours, the doctor on duty, and two reminders for patients — inside a box drawn with * characters.*

Chapter 2 — Data Types

Recap: Every value has a type, which decides how much memory it uses and what you can do with it: `int` for whole numbers, `double` for decimals, `char` for a single character in single quotes, `bool` for true or false, and `std::string` for text in double quotes (with `#include <string>`). Dividing an `int` by an `int` throws the remainder away — `7 / 2` is 3. Use `static_cast<double>(...)` when you need the decimal answer, and `sizeof` to see how many bytes a type uses.

Use only: everything from Module 1, plus variables of type `int`, `double`, `char`, `bool` and `std::string` whose values are written into the code, arithmetic (`+` `-` `*` `/`), `sizeof` and `static_cast`. Not yet: reading input (Module 3), or `if/else` and loops (Module 4).

Activity 1 — Pick the Right Type

DIFFICULTY: FOUNDATIONAL

Scenario: *A student profile holds different kinds of information — and each kind deserves its own type.*

Requirements:

- Declare variables for one student: full name, age, height in metres, the first letter of their house name, and whether they have paid their school fees. Choose the most suitable type for each and give each a value.
- Print every value with a clear label.
- Next to each declaration, write a one-sentence comment justifying your choice of type (for example, why age is an `int` and not a `double`).
- Look at how the `bool` is printed. Add a comment explaining what you see and why.

Activity 2 — Trotro Fare Calculator

DIFFICULTY: CORE

Scenario: *A trotro driver charges the same fare to every passenger and wants to know how the day went.*

Requirements:

- Store the fare per passenger in a `double` (e.g. 8.50 cedis) and the number of passengers on each of three trips in three separate `int` variables (e.g. 18, 17 and 15).
- Calculate and print the total number of passengers and the total takings.
- Store the day's fuel cost in a `double` and print the profit (takings minus fuel).
- Calculate the average number of passengers per trip twice: once as `int / int`, and once using `static_cast<double>`. Print both, and explain in a comment which one is correct and why they differ.

Activity 3 — Memory Map

DIFFICULTY: CHALLENGE

Scenario: Engineers who program small devices — like the microcontroller inside a robot — care exactly how much memory every value takes.

Requirements:

- Use `sizeof` to print the size in bytes of `int`, `double`, `float`, `char`, `bool` and `std::string` on your computer.
- A robot stores 1000 temperature readings. Calculate and print how many bytes they would take as `double`, how many as `float`, and the difference between the two.
- Store 9.99 in an `int` and in a `double` and print both. Then print `static_cast<int>('A')`. Explain both results in comments.
- Finish with a comment block answering: for the robot's readings, would you choose `float` or `double`, and why?

Hint (read after a real attempt): `sizeof(double)` gives the size of ONE double, so the memory for 1000 readings is `1000 * sizeof(double)`. Store each answer in its own variable with a clear name before you print it.

Domain Twist (optional)

EE twist: store a voltage and a resistance and use Ohm's law ($\text{current} = \text{voltage} / \text{resistance}$) to calculate the current. Then repeat with `int` variables (e.g. 12 volts and 1000 ohms) and see integer division give a current of 0 — explain why in a comment.

Biomedical twist: store a patient's weight in kg and a medicine dose in mg per kg, and calculate the total dose. Repeat the calculation with `int` variables and explain in a comment why integer division in a dose calculation could be dangerous.

Chapter 3 — Variables

Recap: A variable is a named box in memory that holds one value of one type. Declare it once, initialise it straight away, and change it as often as you need — unless it is `const`, in which case the compiler refuses any change. Compound assignment (`+=`, `-=`, `*=`) and `++/--` update a variable in place. `std::cin >>` reads one word or number typed by the user, and `std::getline(std::cin, text)` reads a whole line. A variable only exists inside the `{ }` block where it was declared.

Use only: everything from Modules 1-2, plus `const`, `+=`, `-=`, `*=`, `++` and `--`, reading input with `std::cin >>` and `std::getline`, and `{ }` blocks. Not yet: `if/else`, `switch` or loops (Module 4) — your programs still run straight from top to bottom.

Activity 1 — Introduce Yourself

DIFFICULTY: FOUNDATIONAL

Scenario: *This is your first program that holds a real conversation with its user.*

Requirements:

- Ask the user for their full name FIRST, using `std::getline` so that names with spaces work. Then ask for their year of birth and their favourite subject (one word).
- Store the current year in a `const int` and use it to calculate the age they will turn this year.
- Print a friendly sentence using everything they typed, e.g. Hello Ama Mensah! You will turn 15 this year, and your favourite subject is Science.
- Choose clear camelCase names for every variable — someone else should understand your code without asking you.

Activity 2 — Mobile Money Wallet

DIFFICULTY: CORE

Scenario: *A mobile money wallet starts with a balance, receives money and pays for things.*

Requirements:

- Read the starting balance from the user.
- Read three transactions in turn: an amount received, an amount spent on airtime, and an amount spent on a bill. Update ONE balance variable with `+=` and `-=` after each, printing the new balance every time.
- Store a fixed fee for each payment (e.g. 0.50) in a `const double`, and subtract it for each of the two payments.
- Print a final summary: starting balance, total received, total spent including fees, and closing balance. You can't yet stop the balance going below zero — in a comment, say what Module 4 will let you do about that.

Activity 3 — Scope Detective

DIFFICULTY: CHALLENGE

Scenario: *Variables with the same name in different blocks are a classic source of confusing bugs. Investigate how C++ handles them before they catch you out.*

Requirements:

- Write a program with `int count = 10;` in main, then an inner `{ }` block that declares its own `int count = 99;` and prints it, then print count again after the block has closed.
- BEFORE running it, write a comment predicting what each line will print. Then run it and compare. Were you right?
- Inside the inner block, add `int bonus = 5;`. Try to print bonus after the block has closed. Copy the compiler's message into a comment and explain it in your own words, then remove that line so the program compiles again.
- Design one more experiment of your own showing that an inner block CAN read and change a variable declared outside it, and comment on the result.

Hint (read after a real attempt): *A variable declared inside `{ }` is created when the block starts and destroyed when it ends. When two variables have the same name, the inner one hides the outer one until its block ends — the outer one has not changed.*

Domain Twist (optional)

EE twist: read a battery voltage and a bulb's power rating from the user, calculate the current the bulb draws ($\text{current} = \text{power} / \text{voltage}$), then read a number of hours and calculate the energy used ($\text{power} \times \text{hours}$). Biomedical twist: read a patient's full name (a whole line), temperature and pulse, then print a neat observation record that includes the ward name stored in a `const std::string`.

Chapter 4 — Control Structures

Recap: if / else if / else chooses one path — the first true condition wins, so order matters. switch picks a case by exact value (remember the break). while repeats while a condition holds; do-while always runs at least once; for repeats a known number of times. break leaves a loop, continue skips to its next round, and % gives the remainder — perfect for even/odd and every-Nth tests. Always make sure a loop's condition can eventually become false.

Use only: everything from Modules 1-3, plus comparison and logical operators, if / else if / else, switch, while, do-while and for loops, break, continue and %. Not yet: arrays or vectors (Module 5) or your own functions (Module 6) — everything still lives inside main.

Activity 1 — Grade Checker

DIFFICULTY: FOUNDATIONAL

Scenario: *A teacher wants to type a mark and instantly see its letter grade.*

Requirements:

- Read a mark from 0 to 100. If it is outside that range, print an error message instead of a grade.
- Otherwise print the letter grade — A (90 and above), B (80+), C (70+), D (60+) or F (below 60) — using if / else if / else in the correct order.
- Wrap the whole thing in a loop so the teacher can check many marks, stopping when they type -1.
- Test it with at least these marks, writing the output you expect for each in a comment first: 100, 90, 89, 60, 59, 0, 101, -1.

Activity 2 — Times Table Trainer

DIFFICULTY: CORE

Scenario: *A primary school learner is practising times tables and wants a patient study partner.*

Requirements:

- Ask which times table to practise (1 to 12), and keep asking until the number is in range. Then print the whole table from 1 to 12 using a for loop.
- Quiz the learner with five questions from that table (e.g. $7 \times 4 = ?$), read each answer, and print Correct! or Not quite — the answer is 28.
- Keep a score and print it at the end, e.g. You scored 4 out of 5.
- Use a switch on the final score to print a different encouraging message for 5, for 4, for 3, and for anything lower (default).

Activity 3 — Pattern Printer

DIFFICULTY: CHALLENGE

Scenario: *Printing shapes with nested loops is how many programmers first learn to 'see' rows and columns.*

Requirements:

- Read a size n from the user, using a do-while loop that keeps asking until n is between 1 and 20.
- Using nested for loops, print one after another: a solid $n \times n$ square of `*`; a right-angled triangle (one star on the first row, n on the last); and a hollow square (stars on the border only).
- Print n rows in which even-numbered rows are made of `#` and odd-numbered rows of `*`, deciding with `%`.
- No row may be printed with a ready-made string like `"*****"` — every character must come from a loop.

Hint (read after a real attempt): *Let the OUTER loop count the rows and the INNER loop print the characters of one row. In the hollow square, a position is on the border if it is in the first or last row, or the first or last column — combine those four tests with `//`.*

Domain Twist (optional)

EE twist: build a battery charge monitor — read charge percentages until the user types -1, print FULL, GOOD, LOW or CRITICAL for each, and report how many CRITICAL readings there were. Biomedical twist: build a temperature checker — read body temperatures until -1 is entered, classify each as low, normal, fever or high fever, and print the highest temperature seen. In both twists, store the boundaries as clearly named const double values and note in a comment where your boundary values came from.

Chapter 5 — Data Structures

Recap: An array holds a fixed number of values of one type. A `std::vector` holds a list that grows with `push_back` and shrinks with `pop_back`, and always knows its own `size()`. Indexes start at 0, so the last valid index is `size() - 1`. A `std::string` is a sequence of chars you can index and loop over. A struct groups values of DIFFERENT types under one name — and a vector of structs is how real records are stored.

Use only: everything from Modules 1-4, plus arrays, `std::vector` (`push_back`, `pop_back`, `size`), range-based for loops, `std::string` as a sequence (`length()` and `[ ]`), and struct. Not yet: your own functions (Module 6) — all code still lives inside `main`.

Activity 1 — Weekly Rainfall

DIFFICULTY: FOUNDATIONAL

Scenario: *The school weather club records the rainfall, in millimetres, for each day of one week.*

Requirements:

- Store seven daily rainfall values in an array of double — hard-code them, or read them from the user.
- Print each day's value with its day number (Day 1 to Day 7) using a loop.
- Calculate and print the total, the average, the wettest day, and the number of dry days (0 mm).
- In a comment, explain what would happen if your loop went up to index 7, and why.

Activity 2 — Word Inspector

DIFFICULTY: CORE

Scenario: *A spelling app needs to analyse the words that learners type.*

Requirements:

- Read one word from the user. Print its length, its first and last letters, and the word written backwards (loop from the last index down to 0).
- Count and print how many vowels it contains (a, e, i, o and u in lower case are enough).
- Report whether the word is a palindrome — the same forwards and backwards, like level — by comparing letters from both ends inside a loop.
- Keep inspecting words until the user types quit.

Activity 3 — Canteen Orders

DIFFICULTY: CHALLENGE

Scenario: *The school canteen wants to stop writing orders on paper.*

Requirements:

- Define struct `MenuItem` with a `std::string` name and a double price, and fill a `std::vector<MenuItem>` with at least five items.

- Print the menu numbered from 1, then let the customer order by typing item numbers, one at a time, until they type 0. Reject any number that doesn't match an item.
- Store each chosen item in a second vector — the order. When they finish, print the order, its total, and the most expensive item in it.
- Let the customer type -1 to remove the LAST item they ordered (use `pop_back`) — but only if the order isn't empty.

Hint (read after a real attempt): *The customer types 1 for the first item, but its index in the vector is 0. Convert once ($\text{index} = \text{choice} - 1$) and check that the index is between 0 and $\text{size}() - 1$ before you use it.*

Domain Twist (optional)

EE twist: store a list of resistor values in a vector and calculate the total resistance in series (their sum), plus the largest and smallest resistor. Biomedical twist: define struct `Reading` with a `std::string` `time` and an `int` `heartRate`, store a day of readings in a vector, and list every reading above 100 beats per minute.

Chapter 6 — Functions I

Recap: A function packages one task under a name: a return type, a name, a list of parameters and a body. Call it as often as you like with different arguments. A void function does a job and returns nothing. Put a prototype above main and the full definition below it. By default, parameters are COPIES — changing a parameter inside a function never changes the caller's variable.

Use only: everything from Modules 1-5, plus your own functions with parameters and return values, void functions, and prototypes. Parameters are passed by value. Not yet: reference parameters (&), default parameters, overloading or recursion (Module 7).

Activity 1 — Unit Converter Library

DIFFICULTY: FOUNDATIONAL

Scenario: An engineering student keeps converting units by hand. Write each conversion once, as a function, and never do it by hand again.

Requirements:

- Write at least five conversion functions, each taking one double and returning a double — for example kmToMiles, milesToKm, kgToPounds, metresToFeet and cedisToDollars (with the exchange rate stored in a const double).
- Give every function a prototype above main and its full definition below main.
- In main, call each function at least twice with different values and print clearly labelled results.
- Above each prototype, write a one-line comment saying what the function takes and what it returns.

Activity 2 — Menu Calculator

DIFFICULTY: CORE

Scenario: A calculator is a perfect first program built from small, named pieces.

Requirements:

- Write four functions — add, subtract, multiply and divide — each taking two doubles and returning a double.
- Write a void function printMenu() that only prints the menu, and a function double readNumber(std::string prompt) that prints the prompt, reads a number and returns it.
- In main, loop: show the menu, read a choice, read two numbers with readNumber, call the right function using a switch, and print the result. Exit when the choice is 0.
- Before calling divide, check for a zero divisor and print a clear message instead of dividing.

Activity 3 — Refactor the Grade Tracker

DIFFICULTY: CHALLENGE

Scenario: Your Module 5 Student Grade Tracker works — but its main is long and repeats itself. Refactoring means improving a program's structure *WITHOUT* changing what it does.

Requirements:

- Copy your Module 5 project into project-book/module-06/. Run it and paste its output into a comment at the top of the file — this is the behaviour you must keep.
- Move the work into functions: `double average(std::vector<double> scores)`, `double classAverage(std::vector<Student> students)`, `std::string topStudentName(std::vector<Student> students)` and `void printStudent(Student s)`.
- main should now be short enough to read at a glance — mostly calls to your functions.
- Run the program again and confirm the output is identical. In a comment, give one advantage of the new version.

Hint (read after a real attempt): The struct *Student* definition must appear *ABOVE* any prototype that uses it. The compiler reads from top to bottom, so it has to know what a *Student* is before it meets the word *Student* in a parameter list.

Domain Twist (optional)

EE twist: write functions `current(voltage, resistance)`, `seriesResistance(r1, r2)` and `parallelResistance(r1, r2)`, then print a table of results for several resistor pairs.
Biomedical twist: write functions `doseInMg(weightKg, mgPerKg)`, `bmiCategory(bmi)` returning a `std::string`, and `maxHeartRate(age)` (220 minus age), then print a short health summary for three patients.

Chapter 7 — Functions II

Recap: A reference parameter (`int &x`) is another name for the caller's own variable, so changes inside the function are real — use one deliberately when a function must update a variable or hand back more than one result. Default parameters supply values the caller leaves out, and must come at the END of the parameter list. Overloaded functions share a name but differ in their parameters. A recursive function calls itself on a smaller version of the problem, and must always reach a base case.

Use only: everything from Modules 1-6, plus reference parameters (&), default parameter values, function overloading and recursion. Not yet: classes (Module 8).

Activity 1 — Swap, Sort and Summarise

DIFFICULTY: FOUNDATIONAL

Scenario: *Some jobs need a function that changes the caller's variables, or hands back more than one answer.*

Requirements:

- Write `void swapValues(int &a, int &b)` and prove in main that it really swaps two variables.
- Write `void sortThree(int &a, int &b, int &c)` that puts three numbers into ascending order by calling your `swapValues`.
- Write `void summarise(std::vector<double> values, double &minimum, double &maximum, double &average)` that hands back three answers through reference parameters.
- In a comment, explain what would happen to `swapValues` if you removed the & symbols, and why.

Activity 2 — Printing Helpers

DIFFICULTY: CORE

Scenario: *Programs print headings, lines and boxes all the time. A few well-designed helpers make every future program look tidy.*

Requirements:

- Write `void printLine(int length = 30, char symbol = '-')` and call it three ways: with no arguments, with only a length, and with both.
- Write an overloaded function `printValue` with four versions — for an int, a double, a `std::string`, and a `bool` (printing yes or no).
- Write `void printBoxed(std::string text, char border = '*')` that prints the text inside a box exactly the right width for that text (use `text.length()`).
- Use your helpers to print a small report card for one student. In a comment, explain how the compiler decides which `printValue` to call.

Activity 3 — Recursion Lab

DIFFICULTY: CHALLENGE

Scenario: *Recursion feels strange until you have traced it by hand a few times. This lab makes you do exactly that.*

Requirements:

- Write these recursive functions: `void countDown(int n)`, which prints `n` down to 1 and then `Liftoff!`; `int fibonacci(int n)`; `int countChar(std::string s, char target, int index)`, which counts how often `target` appears from `index` onwards; and `void printReverse(std::string s, int index)`, which prints a string backwards.
- For each function, write its base case in a comment, and explain what would happen without it.
- Draw the full call tree for `fibonacci(5)` by hand — on paper or in a comment — and count the calls. Then add an `int &calls` parameter to check your count.
- Write a loop version of `fibonacci` and run both for `n = 30`. Which is faster, and why? Answer in a comment.

Hint (read after a real attempt): *To count calls, add 1 to calls as the very first line of `fibonacci`, and pass the SAME calls variable into both recursive calls. Set it to 0 in main before the first call.*

Domain Twist (optional)

EE twist: write a recursive function that totals the resistance of `n` equal resistors in series, and a function that uses reference parameters to hand back both the current and the power for a given voltage and resistance. Biomedical twist: write a dose function with a default `mg-per-kg` value, and a function that uses reference parameters to hand back the lowest, highest and average of a day's blood-sugar readings.

Chapter 8 — Classes and Objects

Recap: A class bundles data (member variables) with the behaviour that works on that data (methods). Keep the data private and offer public methods that enforce the rules, so an object can never be put into a nonsense state. A constructor runs when an object is created and should leave it valid from the very start. `this->name` means 'my own member', which matters when a parameter has the same name. A class is the blueprint; an object is one real thing built from it.

Use only: everything from Modules 1-7, plus classes with private member variables and public methods, constructors, getters and setters with validation, and `this->`. Not yet: inheritance (Part 2) — each class stands on its own, although one class may hold objects of another class in a `std::vector`.

Activity 1 — Library Book

DIFFICULTY: FOUNDATIONAL

Scenario: *The school library wants to know, for every book, whether it is on the shelf.*

Requirements:

- Write a class `Book` with a private `title`, `author`, `year` and a `bool isBorrowed`.
- Write a constructor that takes the `title`, `author` and `year`. Every new book starts on the shelf.
- Add `borrowBook()` and `returnBook()` methods that refuse — with a clear message — to borrow a book that is already out, or to return one that is already on the shelf.
- Add a getter for every member and a `printInfo()` method. In `main`, create three books and show every rule being enforced.

Activity 2 — Phone Battery Simulator

DIFFICULTY: CORE

Scenario: *A phone's battery percentage must always stay between 0 and 100, however the phone is used.*

Requirements:

- Write a class `Battery` with a private `int percentage`, set by the constructor. If the constructor is given a value below 0 or above 100, bring it back into range.
- Add `use(int minutes)`, which drains 1% for every 3 minutes of use without going below 0, and `charge(int minutes)`, which adds 2% per minute without going above 100.
- Add `getPercentage()` and a `status()` method that returns `Full`, `Good`, `Low` or `Empty` as a `std::string`.
- In `main`, simulate a day with a loop of mixed use and charging, printing the percentage and status after every step. Then try to break the rules from `main` — you shouldn't be able to.

Activity 3 — Football League Table

DIFFICULTY: CHALLENGE

Scenario: *An inter-school football competition needs a league table that updates after every match.*

Requirements:

- Write a class `Team` with a private `name`, `played`, `won`, `drawn`, `lost`, `goalsFor` and `goalsAgainst`, plus methods `points()` (3 for a win, 1 for a draw) and `goalDifference()`.
- Write a class `League` with a private `std::vector<Team>`, an `addTeam(std::string name)` method, and `recordMatch(std::string home, int homeGoals, std::string away, int awayGoals)`, which updates BOTH teams. Reject a match if a team name doesn't exist or a score is negative.
- Add `printTable()`, which prints every team's statistics in neat columns, sorted by points (highest first) using a selection sort or bubble sort you write yourself. When points are level, the better goal difference goes higher.
- In main, add at least four teams, record at least six matches, and print the table after each round.

Hint (read after a real attempt): *recordMatch must change the Team objects stored INSIDE the vector, not copies of them. Loop with an index (`teams[i]`) or use a reference in a range-based for loop (`for (Team &t : teams)`) so your updates are not lost.*

Domain Twist (optional)

EE twist: write a class `Resistor` (resistance in ohms, which must be positive) and a class `Circuit` holding a vector of resistors, with `totalSeries()` and `totalParallel()` methods. Biomedical twist: write a class `Patient` whose temperature and heart-rate setters reject impossible values, and a class `Ward` holding a vector of patients, with a method that lists every patient whose readings need attention.

Extended Capstone — The Complete School Report System

Module 9's own capstone already brings every Part 1 concept together in a Student Records & Grading System. This Extended Capstone is optional and goes further: your system gains a third class, results for several subjects, attendance, much stronger input checking, ranked reports, and a written reflection. Treat it as a portfolio centrepiece, not a required assignment — attempt it once your Module 9 capstone works and all three of its stages are committed. It uses ONLY Part 1 concepts; you don't need anything from Part 2.

Copy your Module 9 capstone into `project-book/extended-capstone/` and extend it there. Each numbered section below is one stage: finish it, test it, and commit it before starting the next.

1. A Subject class

Add a class `Subject` with a private `name` and `std::vector<double>` `scores`, and move the score rules into it: scores outside 0-100 are rejected, `average()` returns 0 when there are no scores yet, and `letterGrade()` returns A to F as before. Each `Student` now holds a `std::vector<Subject>` instead of a single list of scores, with `addSubject(std::string name)` and `addScore(std::string subject, double score)` methods. A student's overall average becomes the average of their subject averages.

2. Attendance

Give each `Student` an attendance record: `days present` and `school days so far`, with `markPresent()` and `markAbsent()` methods, and an `attendancePercentage()` method that never divides by zero. Add a menu option that takes the register for the whole class in one go, asking `Present? (y/n)` for each student in turn.

3. Input you can't break

Every value the program reads must be checked. A menu choice outside the menu, a score outside 0-100, a student or subject name that doesn't exist, and a name that is already taken must each produce a clear message and a chance to try again (use `do-while` loops) — never a crash or a silently wrong result. Write small helper functions such as `int readMenuChoice(int lowest, int highest)` so the checking lives in one place instead of being repeated. (You may assume the user types a number whenever a number is asked for — handling letters typed in place of numbers needs tools from later in the course.)

4. Reports that answer real questions

Add a reports menu with at least four reports: every student ranked by overall average, highest first, using a sort you write yourself; the top student in each subject; how many students earned each letter grade (A to F) in a chosen subject; and every student whose attendance is below 80%. Make each report its own function or method, and make sure no report changes the data — reports only read it.

5. A written reflection (REFLECTION.md)

Add a REFLECTION.md file to your project answering, in your own words, in 200-400 words in total: Which design decision was hardest — for example, deciding which class should own which data? Which bug took you longest to find, and how did you find it? Which Part 1 module helped you most in this project, and why? If you could add one feature you don't yet know how to build, what would it be?

When You're Done

There is no single correct design for this project. Grade yourself (or ask your instructor to) against whether: the program compiles without errors and never crashes on any menu input; every rule holds whatever the user types (no score above 100, no attendance above 100%); the reports are correct — check them by hand for at least one small class; your GitHub history shows at least five commits, one per stage; and REFLECTION.md shows specific, honest thinking rather than general statements. When you're ready for more, Part 2 begins with Module 10 — Pointers & Dynamic Memory.